

Android 第九章第二节教学讲义（广播与系统事件实战）

各位同学，今天咱们继续深入学习 Android 广播和系统事件处理，重点掌握几个实用场景：监听系统分钟变化、网络状态变更，用 AlarmManager 实现定时任务，以及屏幕方向切换、应用切换到桌面或任务列表等操作。这些功能在天气 APP、闹钟 APP、工具类应用中非常常见，咱们用 "场景 + 代码" 的方式，保证小白也能轻松上手！

一、系统分钟到达广播：实时获取时间变化

系统每分钟会发送一次广播（`Intent.ACTION_TIME_TICK`），我们可以监听这个广播实现 "每秒 / 每分钟更新 UI" 的功能（比如电子时钟）。

1. 特点说明

- 这个广播**只能动态注册**（系统限制，静态注册收不到）；
- 每分钟触发一次，无需权限。

2. 实战：实现实时时钟

步骤 1：创建广播接收者

```
public class TimeTickReceiver extends BroadcastReceiver {
    private OnTimeChangeListener listener;
    // 定义回调接口，通知Activity时间变化
    public interface OnTimeChangeListener {
        void onTimeChanged(String time);
    }
    public TimeTickReceiver(OnTimeChangeListener listener) {
        this.listener = listener;
    }
    @Override
    public void onReceive(Context context, Intent intent) {
```

```

if (Intent.ACTION_TIME_TICK.equals(intent.getAction())) {
    // 获取当前时间
    SimpleDateFormat sdf = new SimpleDateFormat("HH:mm:ss", Locale.getDefault());
    String currentTime = sdf.format(new Date());
    // 通过接口回调更新UI
    if (listener != null) {
        listener.onTimeChanged(currentTime);
    }
}
}
}
}

```

步骤 2：在 Activity 中动态注册

```

public class ClockActivity extends AppCompatActivity implements TimeTickReceiver-
.OnTimeChangeListener {
    private TimeTickReceiver timeReceiver;
    private TextView tvTime;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_clock);
        tvTime = findViewById(R.id.tv_time);
    }
    @Override
    protected void onStart() {
        super.onStart();
        // 注册广播接收者
        timeReceiver = new TimeTickReceiver(this);
        IntentFilter filter = new IntentFilter();
        filter.addAction(Intent.ACTION_TIME_TICK); // 分钟变化广播
        registerReceiver(timeReceiver, filter);
        // 初始显示一次时间
        onTimeChanged(new SimpleDateFormat("HH:mm:ss", Locale.getDefault()).format(new-
Date()));
    }
    @Override
    protected void onStop() {
        super.onStop();
    }
}

```

```
// 取消注册
if (timeReceiver != null) {
    unregisterReceiver(timeReceiver);
}
}
// 实现接口方法，更新时间显示
@Override
public void onTimeChanged(String time) {
    tvTime.setText(time);
}
}
```

步骤 3：布局文件（简单显示时间）

```
<TextView
    android:id="@+id/tv_time"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:textSize="30sp" />
```

效果：屏幕上的时间会每秒更新（因为每分钟广播触发时会获取当前精确时间）。

二、系统网络变更广播：监听网络状态

APP 经常需要判断网络是否可用（比如加载数据前），通过监听网络变更广播可以实时获取网络状态。

1. 权限准备

在AndroidManifest.xml中添加网络权限：

```
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
```

2. 实战：监听网络是否连接

步骤 1：创建网络状态接收者

```
public class NetworkReceiver extends BroadcastReceiver {
    private OnNetworkChangeListener listener;
    // 回调接口：通知网络状态变化
    public interface OnNetworkChangeListener {
        void onNetworkChange(boolean isConnected);
    }
    public NetworkReceiver(OnNetworkChangeListener listener) {
        this.listener = listener;
    }
    @Override
    public void onReceive(Context context, Intent intent) {
        // 检查网络是否连接
        boolean isConnected = isNetworkConnected(context);
        // 回调通知
        if (listener != null) {
            listener.onNetworkChange(isConnected);
        }
    }
    // 工具方法：判断网络是否可用
    private boolean isNetworkConnected(Context context) {
        ConnectivityManager cm = (ConnectivityManager) context.getSystemService(Context.
CONNECTIVITY_SERVICE);
        if (cm != null) {
            // 获取所有网络信息（需API 23+）
            NetworkInfo activeNetwork = cm.getActiveNetworkInfo();
            return activeNetwork != null && activeNetwork.isConnected();
        }
        return false;
    }
}
```

步骤 2：在 Activity 中使用

```
public class NetworkActivity extends AppCompatActivity implements NetworkReceiver-
.OnNetworkChangeListener {
    private NetworkReceiver networkReceiver;
```

```

private TextView tvNetworkStatus;
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_network);
    tvNetworkStatus = findViewById(R.id.tv_network_status);
}
@Override
protected void onStart() {
    super.onStart();
    // 注册广播
    networkReceiver = new NetworkReceiver(this);
    IntentFilter filter = new IntentFilter();
    // 网络状态变化的action (API 28前用这个)
    filter.addAction(ConnectivityManager.CONNECTIVITY_ACTION);
    registerReceiver(networkReceiver, filter);
}
@Override
protected void onStop() {
    super.onStop();
    // 取消注册
    if (networkReceiver != null) {
        unregisterReceiver(networkReceiver);
    }
}
// 网络状态变化时更新UI
@Override
public void onNetworkChange(boolean isConnected) {
    if (isConnected) {
        tvNetworkStatus.setText("网络已连接");
        tvNetworkStatus.setTextColor(Color.GREEN);
    } else {
        tvNetworkStatus.setText("网络已断开");
        tvNetworkStatus.setTextColor(Color.RED);
    }
}
}

```

效果：打开 / 关闭 WiFi 或数据流量时，会实时显示网络状态。

三、AlarmManager：实现定时任务

AlarmManager是系统提供的闹钟服务，即使 APP 退出，也能在指定时间触发任务（比如定时提醒、每日打卡提醒）。

1. 核心用法

- 设定定时任务：set()/setRepeating()（重复任务）；
- 取消任务：cancel()；
- 配合广播接收者：定时到了会发送广播，在接收者中处理任务。

2. 实战：3 秒后发送提醒

步骤 1：创建定时任务的广播接收者

```
public class AlarmReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        // 定时到了，显示提醒
        Toast.makeText(context, "定时任务触发！", Toast.LENGTH_LONG).show();
        // 可以在这里执行其他操作（如启动服务、发送通知）
    }
}
```

步骤 2：在 Manifest 中注册接收者

```
<receiver
    android:name=".AlarmReceiver"
    android:enabled="true"
    android:exported="true">
    <intent-filter>
        <action android:name="com.example.ALARM_ACTION" />
    </intent-filter>
</receiver>
```

步骤 3：在 Activity 中设置定时任务

```
public class AlarmActivity extends AppCompatActivity {
    private AlarmManager alarmManager;
    private PendingIntent pendingIntent;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_alarm);
        // 获取AlarmManager实例
        alarmManager = (AlarmManager) getSystemService(Context.ALARM_SERVICE);
        // 创建意图（指向广播接收者）
        Intent intent = new Intent("com.example.ALARM_ACTION");
        pendingIntent = PendingIntent.getBroadcast(
            this,
            0, // 请求码（区分不同任务）
            intent,
            PendingIntent.FLAG_IMMUTABLE // 权限标记（API 31+必填）
        );
        // 点击按钮设置3秒后触发的任务
        findViewById(R.id.btn_set_alarm).setOnClickListener(v -> {
            // 获取3秒后的时间戳
            long triggerTime = System.currentTimeMillis() + 3000;
            // 设置定时任务（API 19+推荐用setExact()保证精确性）
            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
                alarmManager.setExactAndAllowWhileIdle(
                    AlarmManager.RTC_WAKEUP, // 类型：实时时钟，唤醒屏幕
                    triggerTime,
                    pendingIntent
                );
            } else {
                alarmManager.setExact(
                    AlarmManager.RTC_WAKEUP,
                    triggerTime,
                    pendingIntent
                );
            }
            Toast.makeText(this, "3秒后提醒", Toast.LENGTH_SHORT).show();
        });
        // 取消定时任务
    }
}
```

```
findViewById(R.id.btn_cancel_alarm).setOnClickListener(v -> {  
    alarmManager.cancel(pendingIntent);  
    Toast.makeText(this, "已取消定时任务", Toast.LENGTH_SHORT).show();  
});  
}  
}
```

说明：RTC_WAKEUP表示定时时间基于系统时间，且到点会唤醒屏幕（确保用户能看到提醒）。

四、竖屏与横屏切换：监听屏幕方向变化

APP 有时需要根据屏幕方向（竖屏 / 横屏）显示不同布局（比如视频 APP 横屏全屏），可以通过广播或配置文件实现。

1. 方式 1：在 Manifest 中固定屏幕方向

```
<activity  
    android:name=".MainActivity"  
    android:screenOrientation="portrait"> <!-- 固定竖屏（landscape为横屏） -->  
</activity>
```

2. 方式 2：动态监听方向变化（通过广播）

步骤 1：创建方向变化接收者

```
public class OrientationReceiver extends BroadcastReceiver {  
    private OnOrientationChangeListener listener;  
    public interface OnOrientationChangeListener {  
        void onOrientationChanged(String orientation);  
    }  
    public OrientationReceiver(OnOrientationChangeListener listener) {  
        this.listener = listener;  
    }  
    @Override  
    public void onReceive(Context context, Intent intent) {  
        if (Intent.ACTION_CONFIGURATION_CHANGED.equals(intent.getAction())) {
```



```

// 获取当前屏幕方向
int orientation = context.getResources().getConfiguration().orientation;
String result = (orientation == Configuration.ORIENTATION_PORTRAIT) ? "竖屏" : "横屏";
if (listener != null) {
    listener.onOrientationChanged(result);
}
}
}
}
}

```

步骤 2：在 Activity 中注册

```

public class OrientationActivity extends AppCompatActivity implements OrientationReceiver.
OnOrientationChangeListener {
    private OrientationReceiver orientationReceiver;
    private TextView tvOrientation;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_orientation);
        tvOrientation = findViewById(R.id.tv_orientation);
    }
    @Override
    protected void onStart() {
        super.onStart();
        orientationReceiver = new OrientationReceiver(this);
        IntentFilter filter = new IntentFilter(Intent.ACTION_CONFIGURATION_CHANGED);
        registerReceiver(orientationReceiver, filter);
    }
    @Override
    protected void onStop() {
        super.onStop();
        if (orientationReceiver != null) {
            unregisterReceiver(orientationReceiver);
        }
    }
    @Override
    public void onOrientationChanged(String orientation) {
        tvOrientation.setText("当前方向: " + orientation);
    }
}

```

```
}  
}
```

效果：旋转手机时，会实时显示 "竖屏" 或 "横屏"。

五、回到桌面与切换到任务列表（画中画）

1. 回到桌面（不关闭 APP）

```
// 点击按钮回到桌面  
findViewById(R.id.btn_home).setOnClickListener(v -> {  
    Intent intent = new Intent(Intent.ACTION_MAIN);  
    intent.addCategory(Intent.CATEGORY_HOME); // 跳转到桌面  
    startActivity(intent);  
});
```

2. 切换到任务列表（多任务视图）

```
// 点击按钮打开任务列表  
findViewById(R.id.btn_recents).setOnClickListener(v -> {  
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.N) {  
        // Android 7.0+调用系统方法  
        Intent intent = new Intent(Intent.ACTION_RECENT_APPS);  
        startActivity(intent);  
    } else {  
        Toast.makeText(this, "系统版本不支持", Toast.LENGTH_SHORT).show();  
    }  
});
```

3. 画中画模式（视频 APP 常用）

画中画模式允许 APP 在小窗口中运行，同时用户可以操作其他 APP（需 API 24+）。

步骤 1：在 Manifest 中声明支持画中画

```
<activity
    android:name=".VideoActivity"
    android:supportsPictureInPicture="true"
    android:configChanges="screenSize|smallestScreenSize|screenLayout|
orientation">
</activity>
```

步骤 2：进入画中画模式

```
// 点击按钮进入画中画
findViewById(R.id.btn_pip).setOnClickListener(v -> {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        enterPictureInPictureMode(); // 进入画中画模式
    } else {
        Toast.makeText(this, "需要Android 8.0+", Toast.LENGTH_SHORT).show();
    }
});
```

总结

这节课咱们学了 5 个实用的系统事件和广播相关功能：

1. **系统分钟广播**：动态注册ACTION_TIME_TICK，实现实时时钟；
2. **网络变更广播**：监听CONNECTIVITY_ACTION，实时判断网络是否可用；
3. **AlarmManager**：实现定时任务（即使 APP 退出也能触发），适合提醒功能；
4. **屏幕方向切换**：通过ACTION_CONFIGURATION_CHANGED广播监听横竖屏变化；
5. **应用切换操作**：回到桌面、打开任务列表，以及画中画模式的基本用法。

这些功能在实际开发中非常实用，尤其是网络监听和定时任务，几乎所有 APP 都会用到。课后可以做一个 "定时提醒 APP"，结合 AlarmManager 和网络监听，在指定时间提醒用户，巩固今天的知识！有问题随时问～

（注：文档部分内容可能由 AI 生成）